```
09/07/2016  06:50 PM    <DIR>          .
09/07/2016  06:50 PM    <DIR>          ..
09/07/2016  06:50 PM         8,657,920 influx.exe
09/07/2016  06:50 PM        20,148,224 influxd.exe
09/07/2016  06:50 PM             9,825 influxdb.conf
09/07/2016  06:50 PM         8,818,688 influx_inspect.exe
09/07/2016  06:50 PM        11,308,032 influx_stress.exe
09/07/2016  06:50 PM        15,068,672 influx_tsm.exe
               6 File(s)     64,011,361 bytes
```

Starting the InfluxDB server

The InfluxDB service is the executable file named *influxd.exe*. It comes with a default configuration file named *influxdb.conf*, which we will use while starting the server.

To start the server, execute the *influxd.exe* file with the *run* command and pass the configuration file that is shipped as shown below. You could just launch the executable too and, by default, it will execute the *run* command and use the default configuration file, but it is good practice to provide the configuration file as shown below:

```
C:\influxdb-1.0.0-1>influxd run --config c:\influxdb-1.0.0-1\
influxdb.conf
...........
[run] 2016/10/02 11:30:20 InfluxDB starting,
version 1.0.0, branch master, commit
37992377a55fbc138b2c01edd4deffed64b53989
[run] 2016/10/02 11:30:20 Go version go1.6.2, GOMAXPROCS set
to 4
[run] 2016/10/02 11:30:20 Using configuration at: c:\
influxdb-1.0.0-1\influxdb.conf
……
Rest of the output
```

Leave this process running since this is the daemon that will accept requests to insert the data into the database, either from the command line or via the REST API interface that it exposes.

Creating our database

Now that we have our InfluxDB service up and running, the next thing to do is to create our database. You can have multiple datasets (databases) in your InfluxDB installation, which can each be managed separately.

InfluxDB ships with a neat utility named *influx.exe*, which you will find in your installation directory. This is the client console program, also known as the InfluxDB shell that you will use to talk to the InfluxDB server that we started in the previous section. It works on lines similar to popular database clients, where you can connect to database instances, perform both DDL and DML like operations, and more. If you are familiar with the MySQL client program, this will feel very similar to that.

To launch the InfluxDB client, open another command prompt and navigate to your installation directory. Execute the *influx.exe* application without any parameters, as shown below:

```
C:\influxdb-1.0.0-1>influx.exe

Visit https://enterprise.influxdata.com to register for
updates, InfluxDB server management, and monitoring.

Connected to http://localhost:8086 version 1.0.0
InfluxDB shell version: 1.0.0
>
```

Note that we did not specify any parameters, as a result of which the client connects to the InfluxDB daemon running on the localhost. Alternatively, you can provide multiple other parameters like *--host*, *--port*, etc, to connect to InfluxDB instances running on other host machines across the network, whether on your private network or a public network.

We will now create our database that will hold the time series data that we wish to capture. The sample scenario is that of an office, where we would like to capture two kinds of measurements — temperature and sound levels. You will soon see how you can push this data into the InfluxDB data, but for now, let us call this database OfficeMonitoringDB.

To create the database, give the following *CREATE DATABASE <databasename>* command in the InfluxDB client, as shown below:

```
> CREATE DATABASE OfficeMonitoringDB
> show databases
name: databases
---------------
name
_internal
OfficeMonitoringDB
>
```

The first command will create the database. We then execute another command to list the current databases that are present in this instance of InfluxDB. You can see that in addition to its internal database, it also has our newly created *OfficeMonitoringDB* database.

We can now use the DML statements to create some sample records for our reference. The first step is a standard step, to indicate to the client which specific database we will use for the INSERT statements that we will be firing in a while. This is done via the *USE <databasename>* command, as shown below:

```
> USE OfficeMonitoringDB
Using database OfficeMonitoringDB
>
```

We can now use *INSERT* statements to create sample records for our measurements (temperature and sound). The format for inserting a record is as follows: